czii-cryo-et-object-identification

6位解法

Rank	6
Team	tomoon33
Author	tomoon33
Topic ID	561518
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561518
Retrieved with Kaggle CLI on 2026-10-21.

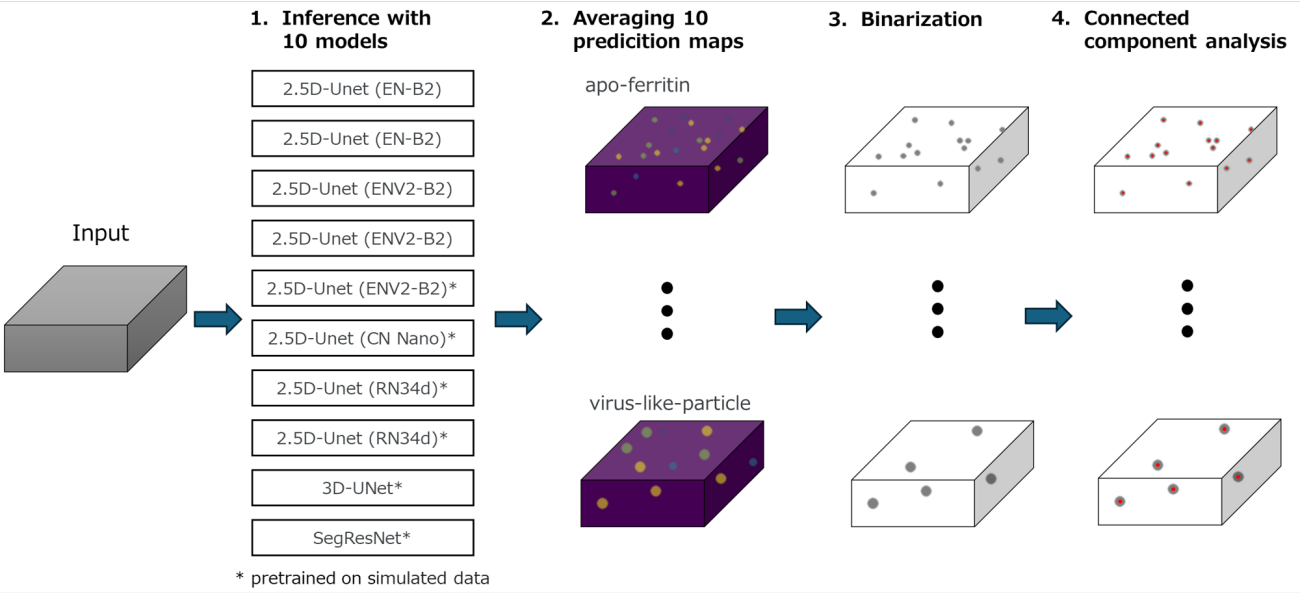
Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

まず、このエキサイティングなチャレンジを開催してくださったコンペ主催者に心から感謝します。その尽力のおかげで、多くを学び、スキルを高める機会を得られました。洞察に満ちた議論と貴重なリソースを提供してくれたKaggle コミュニティにも感謝します。

特に、@hengck23 の discussion と notebook は非常に役立ちました。その洞察と貢献は、このコンペでの私のアプローチを形作るうえで欠かせないものでした。

パイプラインの概要

- MONAI の sliding_window_inference を使って入力ボリュームを固定サイズのサブボリュームに分割し、10 個の異なるモデルで推論する。各モデルは、粒子の種類ごとに粒子の位置を示すマップを予測する。
- 各粒子について、10 個の予測マップを平均する。
- 粒子ごとの閾値を使って、平均したマップを二値化する。
- 二値化したボリュームに連結成分解析を適用し、各連結成分の重心を粒子の位置として計算する。



モデル

この notebook に基づく 2.5D UNet と、MONAI の 3D UNet および SegResNet を使いました。2.5D UNet のエンコーダには、timm の EfficientNet-B2、EfficientNetV2-B2、ConvNeXt-Nano、ResNet34d を使いました。3D UNet と SegResNet は、学習データだけでは当初なかなか良い性能を出せませんでしたが、後で説明するシミュレーションデータで事前学習した後は、2.5D UNet と同程度の精度に達しました。

モデルの多様性を高めるため、事前学習の戦略、データ拡張の手法、その他のハイパーパラメータを少しずつ変えた、合計 10 個のモデルを学習しました。最終提出では、学習データセット全体で学習したモデルを使いました。

学習

前処理

前処理として、まず外れ値を取り除き、その後 min-max 正規化を適用しました。入力ボリュームは 64×128×128 の固定サイズに切り出しました。

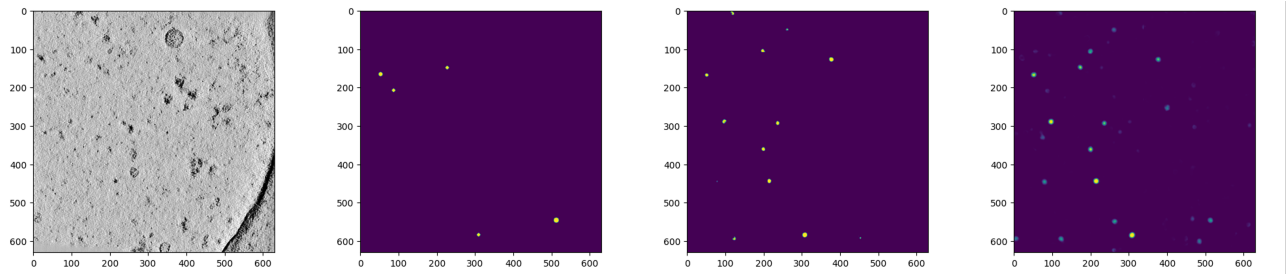
粒子のマスク

学習ラベルとして、各粒子の位置を中心とした二値マスクを生成しました。具体的には、半径 × 0.5 以内の領域を 1、それ以外の領域を 0 にしました。これらのマスクは粒子ごとに別々に作りました。

Loss 関数

最初は BCE と Focal Loss で実験しましたが、これらの loss 関数では、おそらく深刻なクラス不均衡のために、収束が非常に遅くなりました。次に Dice Loss と Tversky Loss を試したところ、学習は大幅に速くなりました。しかし、これらの loss 関数では、モデルの予測が過度に二値的になり、0 か 1 の極端な値しか出力しなくなりました。

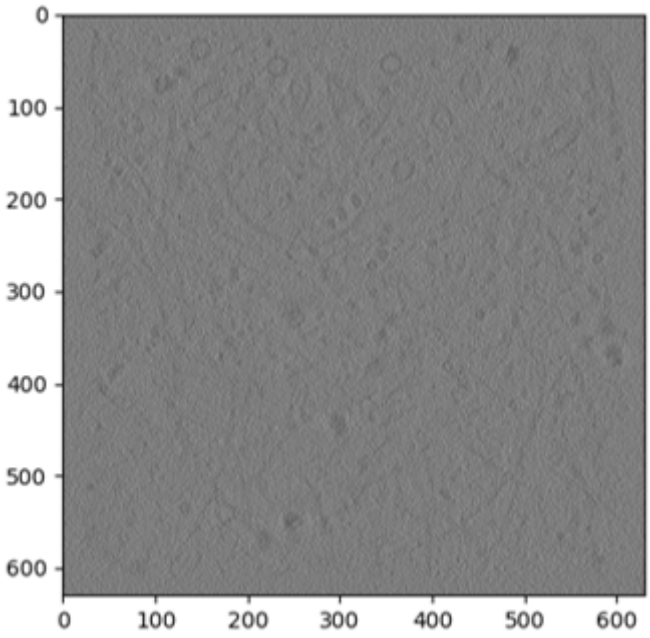
私のパイプラインでは、予測マップを平均してアンサンブルし、その後に閾値をかけて二値化するので、そのような予測値は望ましくありませんでした。これに対処するため、この論文で提案された FocalTversky++ loss を使いました。この loss を使うと、モデルは確信度を反映した予測マップを出力します。



(a) Input (b) Particle mask for thyroglobulin (c) Prediction map for thyroglobulin using Tversky loss (d) Prediction map for thyroglobulin using FocalTversky++ loss

シミュレーションデータでの事前学習

polnet を使って独自のシミュレーションデータを生成し、一部のモデルの事前学習に使いました。粒子を見分けるモデルの能力を高めるため、このコンペの対象粒子と似た形の粒子を含むようにシミュレーションデータを設計しました。



推論

推論には、オーバーラップ率 0.25 の MONAI の sliding_window_inference を使いました。サブボリュームの端に近い予測は不安定になりがちなので、信頼性を高めるために、予測の最も外側の 8% を捨てました。

このコンペの主な課題の一つは、高速な推論を実現することだと感じていました。速度を最適化するため、まず提出前に別の notebook でモデルを TensorRT に変換しました。推論時には、変換済みの TensorRT モデルを読み込み、2枚の T4 GPU で並列処理を行いました。

後処理

アンサンブルした予測マップが得られたら、粒子ごとの閾値で二値化しました。これらの閾値は、最初は交差検証で見積もり、後で leaderboard に基づいて微調整しました。

二値化の後、二値マップに連結成分解析を行い、各連結成分の重心を粒子の位置として取り出しました。

効かなかったこと

- 2段階モデル
- denoised 以外のトモグラムデータの使用

コード

- https://github.com/uchiyama33/czii-6th-place
- https://www.kaggle.com/code/tomoon33/czii-submission-6th-place

補足Q&A

Ma Edward — 質問

おめでとうございます！！！

最終解法の単体モデルのスコアを共有していただけますか。

tomoon33 — 回答

単体モデルの Public LB は 0.757 から 0.766 の範囲でした。

（ただし、単体モデルでは閾値を微調整していないので、最適な閾値を使えばスコアはもう少し高くなる可能性があります。）

Bartley — 質問

興味深いですね、共有ありがとうございます。

Tversky の代わりに FocalTversky++ を使ったとき、CV/LB のスコアがどれくらい変わったか測れましたか。

tomoon33 — 回答

単体モデルのスコアは、CV と LB の両方で約 0.02 上がりました。正確な値は示せませんが、FocalTversky++ を使った場合、アンサンブルによるスコアの改善幅は大幅に大きくなりました。